



Research Intake 2026

Day 27

Image Preprocessing Techniques

A Beginner's Guide for Students

Learning Computer Vision Through Images, Videos, and Artificial Intelligence

Gudsky Research Foundation

Table of Contents

1 Why Preprocessing Matters.....	4
1.1 Garbage In, Garbage Out.....	4
1.2 Preprocessing vs. Augmentation: A Critical Distinction	4
2 Geometric Transformations.....	5
2.1 Resizing and Interpolation.....	5
2.2 Interpolation Method Comparison.....	6
2.3 Cropping, Rotation, and Affine/Perspective Transforms.....	6
2.4 Padding and Aspect-Ratio Preservation	6
3 Photometric Transformations.....	7
3.1 Brightness, Contrast, and Gamma Correction	7
3.2 Histogram Equalization	8
3.3 Color Jitter, Grayscale Conversion, and White Balance.....	8
4 Filtering & Noise Reduction	8
4.1 Convolution as the Basis of Filtering	8
4.2 Blur Filters: Mean and Gaussian	9
4.3 Median and Bilateral Filtering.....	9
4.4 Sharpening Kernels	9
5 Image Normalization & Standardization	10
5.1 Pixel Value Scaling	10
5.2 Per-Channel Mean/Std Normalization.....	10
5.3 Batch Normalization: A Related but Distinct Concept.....	11
6 Data Augmentation for Deep Learning	12
6.1 Purpose: Regularization and Robustness	12
6.2 Classic Augmentations	13
6.3 Advanced Augmentations.....	13
6.4 Domain-Specific Augmentation Constraints	14
7 Preprocessing Pipelines in Practice	14
7.1 Building Reproducible Pipelines	14
7.2 Ensuring Train/Val/Test Consistency	15
7.3 On-the-Fly vs. Precomputed Preprocessing.....	15
7.4 Library Comparison: torchvision vs. Albumentations vs. OpenCV-only.....	15
8 GPU-Accelerated Image Processing.....	16
8.1 Why CPU Preprocessing Becomes a Bottleneck.....	16
8.2 CUDA Fundamentals: Parallelism Across Pixels.....	16
8.3 Three Library Approaches: torchvision-GPU, DALI, and CuPy.....	17
8.3.1 Comparing the Three Approaches	17
8.4 CPU vs. GPU Preprocessing Throughput.....	18
8.5 GPU Memory Considerations	19

9 Real-World Applications & Pitfalls.....	19
9.1 Domain-Specific Preprocessing Considerations.....	19
9.2 Common Pitfalls.....	20
9.3 Debugging Preprocessing Issues: A Practical Checklist	21
10 Research Frontiers.....	21
10.1 Learned and Differentiable Preprocessing.....	21
10.2 Test-Time Augmentation.....	22
10.3 Preprocessing for Vision Transformers vs. CNNs.....	22
Chapter Summary: Key Takeaways	22
Assignment / Practical Project Brief.....	24
A Approach A — torchvision.transforms.v2 (GPU-Native)	24
B Approach B — NVIDIA DALI	24
C Approach C — CuPy (Low-Level).....	25
D Deliverables, Rubric, and Pitfalls	25
E Submission Checklist.....	26

The future of AI is bright, and you can be part of it!

1 Why Preprocessing Matters

Yesterday's material (Day 26) established what a digital image actually is — a tensor of pixel values, structured by a chosen colour space and file format — and surveyed the full landscape of tasks that can be built on top of that representation. This day picks up exactly where that left off: every task in Day 26 §4's taxonomy depends on the input reaching the model in a consistent, well-conditioned form, and preprocessing is the unglamorous but load-bearing step between "we have an image" (Day 26) and "we can do something useful with it" (Day 28 onward). Figure 1 lays out the full pipeline this day covers, section by section.

The Image Preprocessing Pipeline

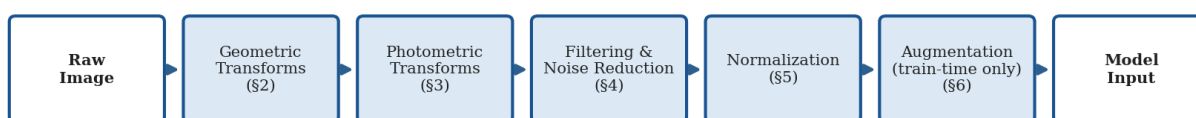


Figure 1. The image preprocessing pipeline this day covers: geometric and photometric transforms, filtering, normalization, and training-time augmentation.

1.1 Garbage In, Garbage Out

The "garbage in, garbage out" principle applies to computer vision with unusual force, because a vision model has no way to distinguish "the object genuinely looks like this" from "the preprocessing pipeline corrupted the input before the model ever saw it." A model trained on images that were accidentally stretched to the wrong aspect ratio will learn to expect stretched objects, and will perform poorly the moment it is deployed against correctly-proportioned images. Because these failures are silent — nothing crashes, no exception is raised, the model simply performs worse than it should — preprocessing bugs are among the most common and most under-diagnosed sources of poor model performance in applied computer vision work.

1.2 Preprocessing vs. Augmentation: A Critical Distinction

It is worth drawing a sharp line between two concepts that are often discussed together but serve entirely different purposes. Preprocessing is fixed and deterministic: the same input image, run through the same preprocessing pipeline, always produces exactly the same output — resizing to 224×224, converting to a specific colour space, normalizing by fixed statistics. Preprocessing must be applied identically at training time and at inference time, because the model has learned to expect its input in exactly the form preprocessing produces. Augmentation, by contrast, is stochastic and training-time only: a random crop, a random rotation, a random colour jitter — applied only during training, specifically to expose the model to controlled variation it would not otherwise see enough of in a finite training set, and never applied at inference time, since inference should show the model a faithful, undistorted view of the real input it needs to make a decision about.

⚠ A Real Case: Silent Train/Inference Preprocessing Mismatch

A team fine-tuned a classifier using torchvision's standard ImageNet normalization statistics (mean [0.485, 0.456, 0.406], std [0.229, 0.224, 0.225]) during training, but their deployed inference service — written independently by a different engineer — normalized images to a simple [0, 1] range with no mean/std adjustment at all.

Validation accuracy during training looked excellent (94%); production accuracy was under 60%. No error was ever raised — the mismatched preprocessing produced valid-looking tensors, just tensors the model had never actually seen the statistical distribution of during training. This class of bug is exactly why Section 5's train/inference consistency point is treated as a hard rule, not a suggestion.

2 Geometric Transformations

2.1 Resizing and Interpolation

Resizing is the most universally applied geometric transformation, since most deep learning architectures require a fixed input resolution. When an image is resized to a different resolution, new pixel values must be computed at positions that did not exist in the original grid — this is what an interpolation method does, and Figure 2 shows the same small image upscaled with three different methods. Nearest-neighbor interpolation simply copies the value of the closest original pixel — fastest, but produces visibly blocky results, especially when upsampling significantly. Bilinear interpolation computes a weighted average of the four nearest original pixels, producing a smoother result at modest additional computational cost. Bicubic interpolation extends this to a weighted combination of the 16 nearest pixels using cubic polynomials, typically producing the sharpest, most visually pleasing result of the three, at the highest computational cost. Area interpolation (a fourth, less commonly discussed method) is specifically well-suited to downsampling, since it averages over the full area each output pixel covers in the original image, avoiding the aliasing artefacts that naive nearest-neighbor downsampling can introduce.

Upsampling the Same Small Image with Different Interpolation Methods

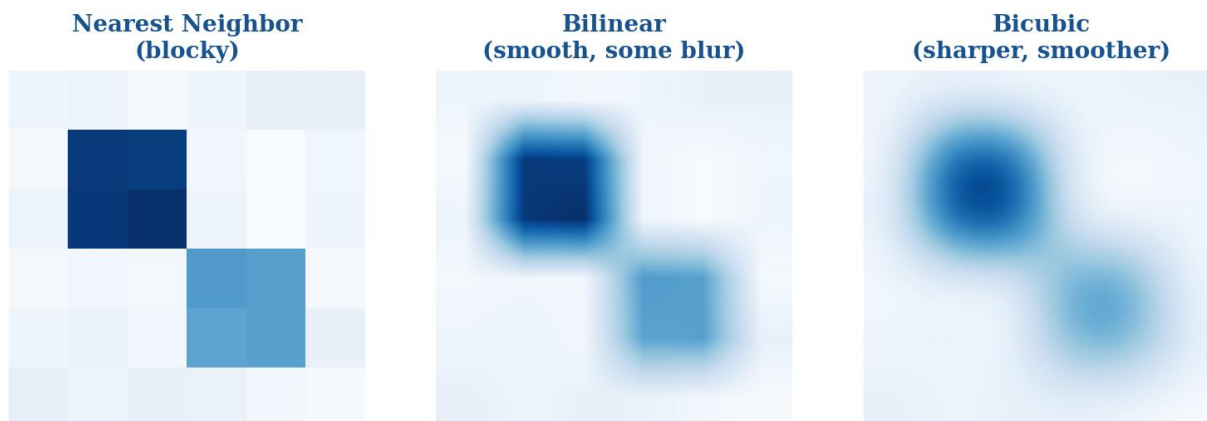


Figure 2. The same small image upsampled with nearest-neighbor, bilinear, and bicubic interpolation — increasing smoothness and computational cost from left to right.

2.2 Interpolation Method Comparison

Method	Quality	Speed	Best Use
Nearest Neighbor	Lowest (blocky)	Fastest	Label/mask resizing (preserves discrete class values)
Bilinear	Moderate	Fast	General-purpose default for photographs
Bicubic	High (sharper)	Moderate	Upsampling where visual quality matters
Area (INTER_AREA)	High for downsampling	Moderate	Significant downsampling (avoids aliasing)

⚠ A Subtle but Important Rule: Never Interpolate Segmentation Masks with Bilinear/Bicubic

A segmentation mask stores discrete integer class IDs (0 = background, 1 = cat, 2 = dog, ...), not continuous intensity values. Bilinear or bicubic interpolation applied to a mask can produce a blended value like 1.4 at a class boundary — a class ID that does not exist and corrupts the label.

Masks must always be resized with nearest-neighbor interpolation, which only ever copies existing (valid) class values — a rule directly relevant to Day 33's segmentation pipelines and worth internalising now, since the mistake is easy to make and does not raise any error.

2.3 Cropping, Rotation, and Affine/Perspective Transforms

Cropping extracts a sub-region of an image — center cropping (the default at inference time, extracting a fixed central region) and random cropping (used as an augmentation, Section 6) are the two most common variants. Rotation rotates an image by a specified angle around a chosen centre point, typically requiring a decision about how to fill the newly-exposed corners (constant fill, edge replication, or reflection). Affine transforms generalise translation, rotation, and scaling into a single linear transformation that preserves parallel lines — useful for correcting camera tilt or standardising object orientation. Perspective transforms are more general still, capable of correcting for the apparent distortion introduced when a planar surface (a document, a whiteboard, a licence plate) is photographed at an angle rather than face-on, and are the geometric mechanism underlying most document-scanning and licence-plate-rectification applications.

2.4 Padding and Aspect-Ratio Preservation

Naively resizing an image to a fixed square resolution distorts its aspect ratio whenever the original image is not already square — a distortion that can meaningfully hurt downstream accuracy, particularly for tasks sensitive to object shape (Day 31's detection, Day 33's segmentation). The standard mitigation is "resize

with padding": scale the image so its longer side matches the target resolution, then pad the shorter side with a constant colour (commonly black or grey) to reach a square. This preserves the true aspect ratio of every object in the image, at the cost of some wasted (padded) pixel area — a trade generally considered worthwhile whenever aspect ratio genuinely matters to the task.

Worked Example: Aspect-Ratio-Preserving Resize

Consider a 1600×900 photograph (16:9 aspect ratio) that needs to become 224×224 for a standard CNN input.

Naive resize (distorting): stretch both dimensions independently to 224×224 — the image is squeezed horizontally by a different factor than vertically, visibly distorting every object's proportions.

Resize-with-padding (preserving): scale so the longer side (1600) becomes 224, giving a scale factor of $224/1600 = 0.14$; the shorter side (900) becomes $900 \times 0.14 = 126$. The result is a 224×126 image, padded with 49 pixels of constant fill on the top and bottom to reach a full 224×224 — every object's true proportions are preserved.

The padded regions carry no real image information, so a model should ideally learn to treat them as background — which happens naturally given enough training examples, but is worth being aware of when interpreting a model's attention/saliency maps (Day 37) near image borders.

3 Photometric Transformations

Where geometric transformations change spatial structure, photometric transformations change pixel intensity values without moving anything spatially — brightness, contrast, colour balance, and related adjustments.

3.1 Brightness, Contrast, and Gamma Correction

Brightness adjustment adds or subtracts a constant offset from every pixel value. Contrast adjustment scales pixel values around a midpoint, spreading (increasing contrast) or compressing (decreasing contrast) the range of intensities present. Gamma correction applies a power-law transformation ($\text{output} = \text{input}^{\gamma}$) that non-linearly remaps intensities — gamma values below 1 brighten shadows disproportionately more than highlights, while values above 1 do the reverse — a correction originally developed to compensate for the non-linear response of CRT displays, but still widely used today to correct for similarly non-linear sensor responses in digital cameras.

Worked Example: Gamma Correction on a Dark Pixel

Consider a pixel with normalized intensity 0.2 (fairly dark) in a backlit photograph where shadow detail needs recovering.

With gamma = 0.5 (brightening shadows): $\text{output} = 0.2^{0.5} \approx 0.447$ — more than double the original brightness, recovering visible shadow detail.

With gamma = 2.0 (darkening shadows further): $\text{output} = 0.2^{2.0} = 0.04$ — the pixel becomes nearly black, the opposite correction, useful for reducing an overexposed image's washed-out appearance.

Note that a bright pixel (0.9) is barely affected by either transform ($0.9^{0.5} \approx 0.949$; $0.9^{2.0} = 0.81$) — gamma correction's effect is concentrated at the dark end of the intensity range, which is precisely why it is the standard tool for shadow recovery rather than a uniform brightness shift.

3.2 Histogram Equalization

Histogram equalization redistributes an image's intensity values so that the resulting histogram is as close to uniform (flat) as possible, which has the effect of enhancing global contrast, particularly useful for images that are predominantly too dark or too bright with intensities crowded into a narrow range. Global histogram equalization applies a single transformation across the entire image, which can over-amplify noise in already-low-contrast regions. CLAHE (Contrast Limited Adaptive Histogram Equalization) addresses this by applying equalization locally, within small tiles of the image, with a clipping limit that prevents any one tile's contrast enhancement from being pushed too far — CLAHE is the standard practical choice whenever histogram equalization is needed on real-world images, and is especially prevalent in medical imaging preprocessing, previewed here ahead of its role in Day 37.

3.3 Color Jitter, Grayscale Conversion, and White Balance

Colour jitter randomly perturbs brightness, contrast, saturation, and hue within small bounds — used almost exclusively as a training-time augmentation (Section 6) rather than a fixed preprocessing step, since its purpose is to expose the model to lighting and colour variation it should learn to be robust to, not to standardise the input. Grayscale conversion, discussed already in Day 26 §2.2, discards colour information entirely and is occasionally used as a deliberate augmentation (randomly converting a fraction of training images to grayscale) to discourage a model from relying too heavily on colour cues that may not generalise. White balance correction adjusts an image's colour temperature to compensate for the colour cast introduced by non-neutral lighting (the orange cast of incandescent lighting, the blue cast of overcast daylight), standardising colour appearance across images captured under different lighting conditions.

4 Filtering & Noise Reduction

4.1 Convolution as the Basis of Filtering

Every filtering operation in this section is, mathematically, a convolution: a small kernel (a matrix of weights) is slid across the image, and at each position, a weighted sum of the pixel values under the kernel

produces the output value at that position — precisely the same mathematical operation as the Sobel edge-detection kernels introduced in Day 26 §5.1, just with a different kernel chosen for a different purpose (smoothing rather than gradient detection). This is a genuinely important throughline for the module: the difference between a classical, hand-designed filter and a single learned convolutional layer in a CNN (Day 29) is only in where the kernel weights come from — hand-chosen here, learned via gradient descent there — the underlying sliding-window weighted-sum operation is identical.

4.2 Blur Filters: Mean and Gaussian

Mean (box) blur replaces each pixel with the simple average of its neighbourhood — cheap to compute but produces a somewhat unnatural, "boxy" blur with visible directional artefacts. Gaussian blur weights the neighbourhood by a 2D Gaussian function, giving nearby pixels more influence than distant ones within the kernel — producing a smoother, more visually natural blur, and it is the standard choice both as a genuine noise-reduction step and as the smoothing sub-step inside the Canny edge detector (Day 26 §5.1).

4.3 Median and Bilateral Filtering

Median filtering replaces each pixel with the median (not mean) value of its neighbourhood — a non-linear operation specifically effective against salt-and-pepper noise (isolated, extreme-valued corrupted pixels), because a median is robust to outliers in a way a mean is not: a single wildly-incorrect pixel value in the neighbourhood barely shifts the median, but can substantially shift the mean. Bilateral filtering is an edge-preserving smoothing filter: it blurs within regions of similar intensity but avoids blurring across strong intensity boundaries (edges), by weighting each neighbour's contribution by both its spatial distance and its intensity similarity to the centre pixel — useful whenever noise reduction is needed without sacrificing the sharp object boundaries that later stages of a pipeline (Day 28's feature detection, Day 31's detection) depend on.

Worked Example: When Filtering Helps vs. Hurts Downstream Performance

Helps: a noisy low-light security camera feed, where Gaussian or bilateral pre-filtering removes sensor noise that would otherwise be mistaken for spurious edges by a downstream classical feature detector (Day 28).

Hurts: aggressively blurring images before training a fine-grained texture classifier (fabric type, skin lesion pattern) — the blur destroys exactly the high-frequency detail the task depends on, and the model's achievable accuracy drops correspondingly.

The general rule: filtering should remove variation the task does not care about (sensor noise) without removing variation the task does care about (task-relevant texture and detail) — a judgment call that depends entirely on the specific downstream task, not a universal default to apply everywhere.

4.4 Sharpening Kernels

Sharpening filters do the conceptual opposite of blurring: they amplify high-frequency detail (edges, fine texture) rather than suppressing it, typically implemented as an "unsharp mask" — subtracting a blurred version of the image from the original, scaling the resulting difference (which isolates high-frequency content), and adding it back to the original. Sharpening is used comparatively sparingly in deep learning

preprocessing pipelines relative to the classical imaging world, since a CNN with sufficient training data can typically learn to be robust to moderate blur directly, but it remains relevant in specific applications — restoring some clarity to images degraded by a lossy compression pipeline, for instance.

Filter	Effect	Robust to Outlier Pixels?	Preserves Edges?
Mean (box) blur	Uniform smoothing	No	No
Gaussian blur	Weighted smoothing	No	No
Median filter	Outlier-robust smoothing	Yes	Partially
Bilateral filter	Edge-aware smoothing	Somewhat	Yes
Unsharp mask	Detail amplification	N/A (sharpens)	Enhances edges

5 Image Normalization & Standardization

5.1 Pixel Value Scaling

Raw pixel values in the 0–255 range are rarely fed directly into a neural network. The simplest normalization scales values into the $[0, 1]$ range by dividing by 255; a slightly less common alternative scales into $[-1, 1]$. Both put pixel values on a scale more numerically compatible with how neural network weights are typically initialised and with how gradient-based optimisers (covered in Day 29) are tuned to behave, avoiding the training instability that can result from feeding a network raw, large-magnitude input values.

5.2 Per-Channel Mean/Std Normalization

The standard practice for models using a pretrained backbone (Day 26 §6.3) goes a step further: after scaling to $[0, 1]$, each channel is normalized by subtracting that channel’s mean and dividing by its standard deviation, using statistics computed across the original pretraining dataset. For ImageNet-pretrained models, this means the now-familiar mean $[0.485, 0.456, 0.406]$ and standard deviation $[0.229, 0.224, 0.225]$, seen already in the torchvision code example in Day 26 §7. Figure 3 shows the effect of this transformation on a pixel value distribution: it centres the distribution around zero and rescales its spread, matching the statistical distribution the pretrained backbone’s weights were originally optimised against.

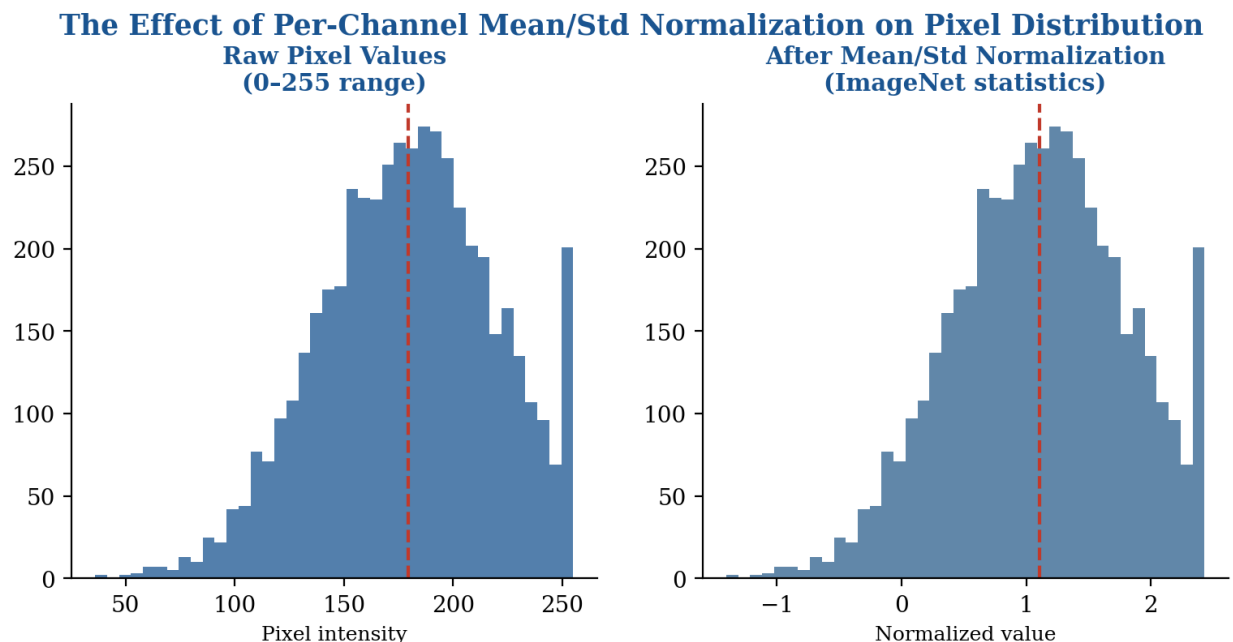


Figure 3. Per-channel mean/std normalization re-centres and rescales the pixel value distribution to match the statistics a pretrained backbone expects.

The critical practical point, worth repeating because it is one of the single most common sources of the silent bug described in Section 1.2's callout: these normalization statistics must be computed once (from the pretraining or original training dataset) and then applied identically, using those exact fixed numbers, at both training and inference time. Using the test/inference batch's own statistics instead — a subtle mistake sometimes made when adapting code — introduces a data-dependent transformation that does not match what the model learned during training.

5.3 Batch Normalization: A Related but Distinct Concept

It is worth flagging, ahead of Day 29's full treatment, that batch normalization is a related but conceptually distinct idea from the input normalization covered in this section. Input normalization (Sections 5.1–5.2) is applied once, to the raw input image, before it ever enters the network. Batch normalization is a layer inside the network itself, normalizing the activations flowing between layers during training — a technique aimed at stabilising and accelerating training dynamics, not at preparing the raw input. The two concepts share the word "normalization" and a similar mean/variance-based mechanism, but operate at entirely different points in the pipeline, and conflating them is a common source of confusion for newcomers.

✎ Worked Example: The Forgotten-Normalization Failure Mode

A model is trained with standard ImageNet mean/std normalization, achieving 91% validation accuracy.

At deployment, an engineer writes a new inference script and forgets the normalization step entirely, feeding the model raw $[0,1]$ -scaled pixels instead of the mean/std-normalized values it was trained on.

The input tensor still has the right shape and a plausible-looking value range, so no error is raised anywhere in the pipeline — but accuracy silently collapses to near-random, because every pixel value the model receives now sits in a completely different statistical range than anything seen during training.

This is precisely why Section 1.2's "preprocessing must be identical at train and inference time" rule is treated as non-negotiable throughout this module.

6 Data Augmentation for Deep Learning

6.1 Purpose: Regularization and Robustness

Data augmentation artificially expands the effective diversity of a training set by applying random, label-preserving transformations to training images on the fly — a different random transformation each time an image is seen, across training epochs. Its purpose is twofold: regularization (reducing overfitting, since the model never sees the literal same pixel values twice, discouraging it from memorising training examples) and robustness (exposing the model to controlled variation — different scales, orientations, lighting — that a real deployment will encounter, but that a finite training set may under-represent).

Common Data Augmentation Operations Applied to the Same Image

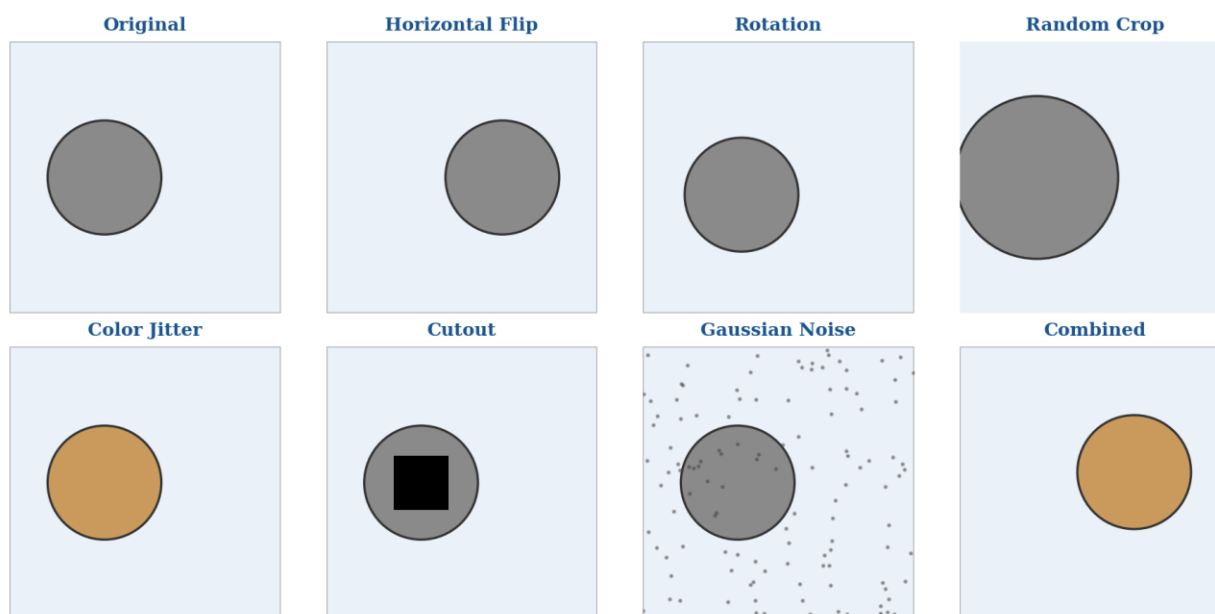


Figure 4. A representative set of augmentation operations applied to the same image — each is a random, label-preserving transformation applied at training time only.

6.2 Classic Augmentations

Horizontal flipping mirrors an image left-to-right — valid for most natural object categories (a mirrored cat is still recognisably a cat) but invalid for tasks where left/right orientation is semantically meaningful (text-containing images, where flipping renders text unreadable and semantically backwards; certain medical imaging orientations, discussed further below). Random rotation and random cropping, illustrated in Figure 4, expose the model to variation in object orientation and framing. Colour jitter (Section 3.3) exposes the model to lighting and colour variation.

6.3 Advanced Augmentations

Beyond these classic operations, several more sophisticated augmentation techniques have become standard in modern training pipelines. Cutout randomly masks a square region of the image to a constant value (typically black), forcing the model to make correct predictions even when part of the object is occluded — encouraging the model to use distributed evidence across the whole object rather than over-relying on one especially discriminative patch. Mixup blends two training images and their labels together in proportion (a weighted average of both the pixel values and the one-hot label vectors), producing a training signal that encourages the model to behave more linearly between classes and has been shown to improve both accuracy and calibration. CutMix combines the spatial patching idea of Cutout with the label-blending idea of Mixup: a rectangular patch from one image is pasted into another, with the label blended in proportion to the pasted area. RandAugment and AutoAugment take a different approach entirely: rather than a human manually choosing which augmentations to apply and how strongly, these methods search over a space of augmentation policies (which operations, in what combination, at what magnitude) to find a policy that empirically maximises validation performance — RandAugment simplifies this search considerably by using only two hyperparameters (number of operations to apply, and a shared magnitude for all of them), making it far cheaper to tune than the original AutoAugment’s full reinforcement-learning-based search.

Augmentation	Mechanism	Primary Benefit
Cutout	Mask a random square region	Robustness to occlusion; distributed feature use
Mixup	Blend two images + labels	Smoother decision boundaries; better calibration
CutMix	Paste a patch + blend labels proportionally	Occlusion robustness + Mixup-style regularization
RandAugment	Randomly sample from an operation set	Strong results with minimal tuning
AutoAugment	Search-optimised augmentation policy	Best possible policy, at high search cost

Research Spotlight: How Much Does Advanced Augmentation Actually Help?

CutMix's original paper (Yun et al., 2019) reported a top-1 accuracy improvement of roughly 2–3 percentage points on ImageNet classification over a strong baseline with only classic augmentation — a substantial gain for a change that requires no architecture modification at all.

RandAugment (Cubuk et al., 2019) matched or exceeded the accuracy of the far more expensive, search-based AutoAugment while requiring orders of magnitude less search compute — a good illustration of the broader pattern that a well-chosen simple method can match a more complex one once the complex method's search space is well understood.

6.4 Domain-Specific Augmentation Constraints

Not every augmentation is valid for every domain, and applying an inappropriate augmentation can silently corrupt training labels rather than helping. Text-containing images should not be horizontally flipped, since flipped text becomes unreadable and semantically wrong. Medical imaging (a full preview of Day 37's dedicated treatment) imposes its own constraints: a chest X-ray should generally not be vertically flipped, since anatomical orientation (heart on the left, for instance) is diagnostically meaningful, and colour jitter is largely inapplicable to greyscale radiographic modalities in the first place. The general principle: an augmentation is valid only if it preserves the property the label actually describes — a rule simple to state but easy to violate by applying a generic augmentation pipeline copied from an unrelated domain without reconsidering it.

Looking forward, it is worth flagging the conceptual link to Day 28: augmentation artificially exposes a model to the scale, rotation, and illumination variation a real deployment will encounter, which is exactly the variation Day 28's feature descriptors (SIFT, ORB) are hand-engineered to be invariant to. Augmentation is the learned-feature era's answer to the same underlying problem that classical descriptors solved by explicit design — two different eras of the field converging on the same underlying requirement.

7 Preprocessing Pipelines in Practice

7.1 Building Reproducible Pipelines

In practice, the individual operations covered in Sections 2 through 6 are chained together into a single, reusable pipeline object, most commonly via torchvision's Compose pattern or the equivalent in Albumentations (introduced in Day 26 §7).

```
import torchvision.transforms.v2 as T

# Training pipeline: preprocessing (fixed) + augmentation (stochastic)
train_transform = T.Compose([
    T.RandomResizedCrop(224),          # geometric ($2) + augmentation ($6)
    T.RandomHorizontalFlip(p=0.5),     # augmentation ($6)
    T.ColorJitter(0.2, 0.2, 0.2, 0.05), # photometric augmentation ($3)
    T.ToTensor(),
```

```

        T.Normalize(mean=[0.485, 0.456, 0.406],
                    std=[0.229, 0.224, 0.225]), # $5
    ])

# Inference / validation pipeline: preprocessing ONLY, no augmentation
eval_transform = T.Compose([
    T.Resize(256),
    T.CenterCrop(224),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406],
                std=[0.229, 0.224, 0.225]),
])

```

7.2 Ensuring Train/Val/Test Consistency

Notice, in the code above, that the training and evaluation pipelines share identical resizing, cropping-to-final-size, and normalization steps — the only difference is that the training pipeline additionally applies stochastic augmentation. This structural pattern — same deterministic preprocessing, different (absent) augmentation — is the correct default for essentially every deep learning vision pipeline, and deviations from it are the single most common source of the train/inference mismatch bugs discussed in Sections 1.2 and 5.2.

7.3 On-the-Fly vs. Precomputed Preprocessing

Preprocessing can be applied on-the-fly (recomputed fresh every time an image is loaded during training, as in the Compose example above) or precomputed once and cached to disk. On-the-fly processing is essential for stochastic augmentation, which must differ on every access by definition, but it is also essential for any deterministic preprocessing step whose parameters might later change (a different target resolution, say) — precomputing and caching a fixed-resolution version of a dataset locks in that choice and requires reprocessing the entire dataset if it needs to change. Precomputing is worthwhile when a genuinely expensive, non-stochastic step (a slow but essential deterministic normalization pass, for instance) would otherwise be needlessly repeated identically on every single training epoch.

7.4 Library Comparison: torchvision vs. Albumentations vs. OpenCV-only

Library	Strength	Weakness
torchvision.transforms	Native PyTorch integration; simple API	Narrower augmentation catalogue than Albumentations
Albumentations	Very broad, fast, composable augmentation set	An additional dependency beyond the core DL framework
OpenCV-only (manual)	Maximum control; no extra dependency	More code to write and maintain per operation

8 GPU-Accelerated Image Processing

8.1 Why CPU Preprocessing Becomes a Bottleneck

At small scale, running the preprocessing pipeline of Section 7 on the CPU while training on the GPU is entirely adequate. At larger scale — a large dataset, a fast GPU, or both — this arrangement can become the limiting factor: if the GPU can process a batch faster than the CPU can prepare the next one, the GPU sits idle waiting for data, and total training throughput is bottlenecked by CPU preprocessing speed rather than by the GPU's actual compute capacity. This imbalance has become increasingly common precisely because GPU compute has advanced faster than typical CPU data-loading throughput, motivating the GPU-accelerated preprocessing approaches this section covers.

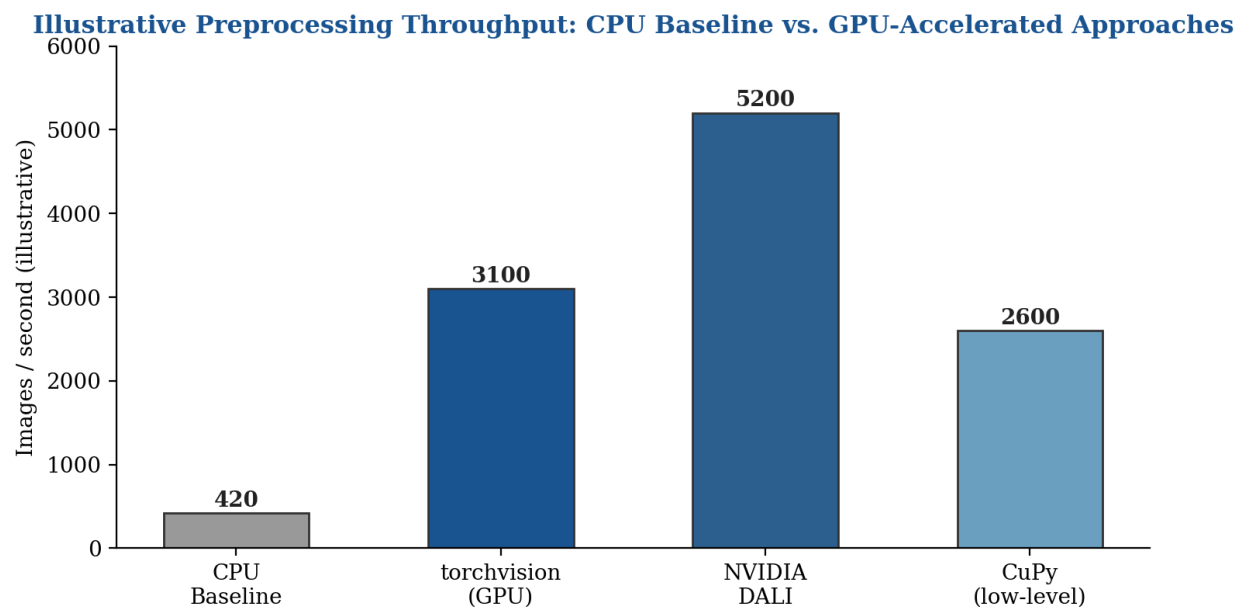


Figure 5. Illustrative preprocessing throughput comparison: GPU-accelerated approaches can substantially outpace a CPU-only baseline, though exact figures vary heavily with hardware, image size, and pipeline complexity.

8.2 CUDA Fundamentals: Parallelism Across Pixels

The reason GPU acceleration helps so much for image preprocessing specifically is that nearly every operation covered in Sections 2 through 6 — resizing, colour conversion, normalization, filtering — is embarrassingly parallel: the computation at one pixel (or small neighbourhood of pixels) does not depend on the result at any other pixel, so all of them can, in principle, be computed simultaneously. A GPU's architecture, with thousands of simple cores designed for exactly this kind of massively parallel, identical-operation-per-data-element workload, is a natural fit — the same underlying hardware advantage that makes GPUs well suited to the convolution operations inside a CNN (Day 29) applies equally well to preprocessing operations performed before the CNN ever runs.

8.3 Three Library Approaches: torchvision-GPU, DALI, and CuPy

Three distinct libraries offer GPU-accelerated preprocessing, differing primarily in their level of abstraction, illustrated in Figure 6. `torchvision.transforms.v2` offers GPU-native execution as a comparatively recent addition — the same familiar Compose-based API from Section 7, but operating

directly on GPU tensors rather than CPU arrays, making it the easiest of the three to integrate into an existing PyTorch training loop with minimal code change. NVIDIA DALI (Data Loading Library) is a purpose-built, production-grade data pipeline library that goes further: it overlaps preprocessing computation with model training computation across separate CUDA streams, so that the next batch is being prepared on the GPU while the current batch is being used for a training step, minimising GPU idle time in a way that simply running transforms on the GPU does not automatically achieve on its own. CuPy is a NumPy-compatible array library that executes on CUDA — the lowest-level of the three, giving direct, transparent visibility into exactly what "GPU-accelerated" means at the level of individual array operations, at the cost of needing to implement higher-level operations (a full augmentation pipeline, say) manually rather than relying on a pre-built transform catalogue.

Abstraction Level:

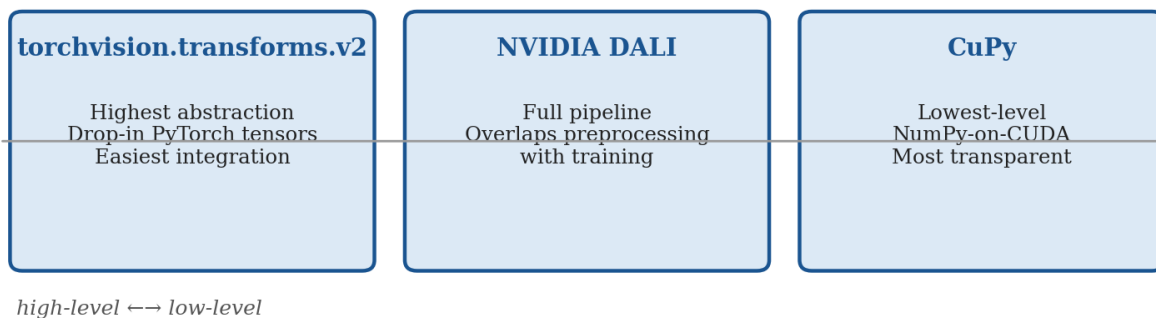


Figure 6. Three GPU-accelerated preprocessing approaches, ordered by abstraction level — higher abstraction trades some control and transparency for ease of integration.

8.3.1 Comparing the Three Approaches

Approach	Abstraction	Setup Complexity	Best-Fit Use Case
torchvision.transforms.v2 (GPU)	High	Low	Fastest path to GPU speedup in an existing PyTorch loop
NVIDIA DALI	Medium (full pipeline)	High	Production training at scale; maximum throughput
CuPy	Low (array-level)	Medium	Understanding the underlying mechanism; custom ops

This comparison table sets up directly the three-way benchmarking exercise this day's practical project, PP19, requires — rather than choosing a single "winner" in the abstract, the assignment asks you to measure the actual throughput and setup trade-offs on your own hardware and pipeline.

Use Case: When DALI's Complexity Pays Off

A team training a large image classifier across 8 GPUs for several days found that CPU-based data loading left each GPU idle roughly 30% of the time, waiting for the next batch — a direct instance of Section 8.1's bottleneck.

Switching to DALI's GPU-resident, pipeline-overlapped loading (this section) eliminated nearly all of that idle time, reducing total training wall-clock time by close to 25% — at the cost of a genuinely more involved setup than either torchvision-GPU or CuPy, illustrating exactly the setup-complexity-vs-throughput trade-off this day's assignment asks you to quantify for yourself.

8.4 CPU vs. GPU Preprocessing Throughput

It is worth being precise about where the GPU advantage actually comes from and where it can disappear. The theoretical throughput advantage of GPU-based preprocessing is largest for large batch sizes and computationally heavier operations (bicubic resizing, complex augmentation chains), where the parallelism described in Section 8.2 has the most work to exploit per kernel launch. For very small batch sizes, the fixed overhead of transferring data from CPU (host) memory to GPU (device) memory can dominate the total time, in some cases making GPU preprocessing no faster, or even slower, than CPU preprocessing — a pitfall explicitly flagged in this day's assignment brief, and a reason why PP19 asks you to benchmark across a few different batch sizes rather than reporting a single number.

Worked Example: Estimating Speedup and the Point of Diminishing Returns

Suppose a CPU preprocessing pipeline processes 420 images/second, and the same pipeline on a GPU (torchvision-GPU) processes 3,100 images/second at batch size 256 — the illustrative figures from Figure 5.

Speedup = $3,100 / 420 \approx 7.4\times$ at this batch size — a substantial, clearly worthwhile gain.

Now suppose the same GPU pipeline is measured at batch size 4: host-to-device transfer overhead is paid on almost every batch, and measured throughput might drop to only 500 images/second — barely faster than the CPU baseline, despite running on identical hardware.

This is exactly why Section 8.4's point about batch-size dependence matters in practice, and why PP19 explicitly requires benchmarking across multiple batch sizes rather than reporting a single favourable number that could be cherry-picked from the large-batch regime alone.

8.5 GPU Memory Considerations

A subtlety specific to GPU-accelerated preprocessing, with no CPU equivalent: preprocessing operations running on the GPU compete for the same limited GPU memory that the model itself, its activations, and its gradients also need during training. A preprocessing pipeline that holds an entire large batch of high-resolution images on-device simultaneously, or that allocates large intermediate buffers for a complex

augmentation chain, can leave insufficient memory headroom for the training step that follows immediately after, producing out-of-memory errors that appear to originate from model training but actually originate from preprocessing's memory footprint. This is one of the genuine costs behind DALI's (Section 8.3) more careful, pipeline-aware memory management, and a factor worth measuring explicitly — not just throughput — when comparing the three approaches for PP19.

9 Real-World Applications & Pitfalls

9.1 Domain-Specific Preprocessing Considerations

Different application domains impose distinct preprocessing requirements shaped by their sensor characteristics and task demands. Satellite imagery is frequently multi-spectral (more than three channels, spanning wavelengths beyond visible light) and requires careful per-band normalization, since different spectral bands can have very different intensity distributions. Medical imaging (Day 37) frequently involves higher bit-depth data (Day 26 §2.1) and modality-specific normalization conventions — CT scan intensities, for instance, are conventionally expressed in Hounsfield units with a fixed physical meaning, and windowing (clipping to a diagnostically relevant intensity range before display or model input) is a domain-specific preprocessing step with no equivalent in natural-image pipelines. Low-light security footage frequently requires denoising and brightness/gamma correction (Sections 3.1, 4) substantially more aggressive than what a well-lit natural photograph needs, and can benefit from CLAHE (Section 3.2) to recover contrast in near-black regions.

Preprocessing at Scale: Satellite Imagery

Agricultural monitoring pipelines processing multi-spectral satellite imagery (e.g., Sentinel-2's 13 spectral bands) must normalize each band independently, since near-infrared reflectance and visible-light reflectance occupy entirely different, non-comparable intensity ranges for the same physical land area.

Applying a single, shared normalization statistic across all bands — the kind of shortcut that works acceptably for standard 3-channel RGB — would badly distort the relative signal across bands and measurably degrade downstream crop-health classification accuracy.

Use Case: Preprocessing for Autonomous Vehicle Perception

Autonomous vehicle camera pipelines must preprocess frames within a strict latency budget (often under 30 milliseconds end-to-end) while handling an enormous dynamic range — direct sunlight, tunnel darkness, oncoming headlight glare — far beyond what a typical consumer photography pipeline needs to tolerate.

This combination of hard real-time constraints (favouring GPU-accelerated preprocessing, §8) and extreme lighting variation (favouring aggressive, well-tuned photometric correction, §3) makes autonomous driving one of the most demanding practical settings for everything covered in this day, and a useful example to keep in mind when Day 31's detection content returns to autonomous driving as an application area.

9.2 Common Pitfalls

Several preprocessing pitfalls recur often enough across real projects to be worth naming explicitly. Data leakage via preprocessing computed on the full dataset before the train/test split is a particularly insidious version of a more general data-leakage problem: if normalization statistics, or any other preprocessing parameter, are computed from the combined train+test data rather than from the training set alone, information about the test set has effectively leaked into the training pipeline, inflating reported test performance in a way that will not replicate on genuinely new production data. Over-aggressive augmentation destroying labels was discussed already in Section 6.4 — an augmentation strong enough to make a human annotator uncertain of the correct label is very likely too strong for the model as well. Normalization mismatches between training and inference (Section 5.2’s worked example) remain, empirically, one of the most common silent failure modes in deployed computer vision systems, precisely because they produce no error and only a quiet, hard-to-diagnose accuracy drop.

Preprocessing Pitfall: Data Leakage via Global Normalization

A team computed per-channel mean/std normalization statistics from their entire dataset (train + validation + test combined) before performing the train/test split — a seemingly innocuous, common-looking line of preprocessing code.

Because the test set’s pixel statistics subtly influenced the normalization applied to the training set, reported test accuracy was optimistic relative to genuinely unseen production data — the fix is to always compute normalization statistics from the training split alone, then apply those fixed values, unchanged, to validation, test, and eventual production inference.

Pitfall	Symptom	Fix
Global normalization statistics	Optimistic test accuracy vs. production	Compute stats from training split only
Over-aggressive augmentation	Model underperforms; labels no longer valid	Check a human can still confidently label augmented samples
Train/inference preprocessing mismatch	Large accuracy drop with no errors raised	Diff the two code paths line-by-line (§9.3)
Bilinear/bicubic mask resizing	Corrupted segmentation labels at boundaries	Always use nearest-neighbor for masks (§2.2)

9.3 Debugging Preprocessing Issues: A Practical Checklist

Because preprocessing bugs are silent by nature (Section 1.1), a systematic debugging checklist is more useful than trying to spot a problem by inspection alone. When a model performs far worse in production than validation accuracy would predict, the following checks, in this order, catch the large majority of real-world cases:

- **Visualise a batch immediately before it enters the model** — plot a few images from the exact tensor the model receives, not the raw file on disk, to catch colour-channel swaps (§2.4), unexpected cropping, or corrupted normalization at the point that actually matters.
- **Diff the training and inference preprocessing code directly** — line by line, not by memory of what each "should" do — since Section 7.2's consistency requirement is violated more often by code drift between two separately-maintained scripts than by any single conceptual misunderstanding.
- **Confirm normalization statistics are hard-coded constants, not recomputed at inference time** — per Section 5.2, using batch-local statistics instead of fixed training-set statistics is a common, easy-to-miss variant of this bug.
- **Check mask/label interpolation method separately from image interpolation method** — per Section 2.2's nearest-neighbor rule, since image and mask resizing are frequently handled by different code paths that can silently drift out of sync.
- **Confirm augmentation is disabled for validation and inference** — a stochastic transform accidentally left active outside training will produce inconsistent, non-reproducible predictions for the same input.

10 Research Frontiers

10.1 Learned and Differentiable Preprocessing

A growing body of research treats preprocessing itself as learnable rather than fixed. Differentiable image processing layers replace a hand-chosen fixed transformation (a specific denoising filter, a specific colour correction) with a small learnable module trained jointly, end-to-end, alongside the downstream task model — allowing the "preprocessing" the model receives to be optimised directly for task performance rather than for a human's independent notion of visual quality, blurring the classical preprocessing/model boundary this day has otherwise treated as a clean separation.

Research Spotlight: Preprocessing as Part of the Model

Differentiable ISP (Image Signal Processor) research replaces the traditional fixed camera preprocessing pipeline — demosaicing, white balance, tone mapping, all normally hand-tuned by camera engineers — with learnable modules trained end-to-end against the downstream task's actual loss function.

The practical implication for this module's broader arc: the classical/deep-learning boundary Day 26 §6 drew so sharply is, at the research frontier, increasingly dissolving — preprocessing, feature extraction, and prediction are converging toward a single, jointly-optimised system rather than a pipeline of independently-designed stages.

10.2 Test-Time Augmentation

Test-time augmentation (TTA) applies the logic of Section 6's augmentation in reverse: rather than augmenting only during training, several augmented versions of a single test image (a few crops, a horizontal flip) are each passed through the trained model at inference time, and the resulting predictions are averaged. This trades increased inference compute (multiple forward passes per image) for a modest, fairly reliable accuracy improvement, since averaging over several slightly different views of the same image reduces the influence of any one view's particular noise or framing.

10.3 Preprocessing for Vision Transformers vs. CNNs

Vision transformers (previewed in Day 26 §10.2 and covered fully in Day 30 and Day 38) introduce preprocessing considerations with no CNN equivalent, because they operate on an image divided into a fixed grid of patches rather than processing the full spatial grid convolutionally. Patch size and image resolution together determine the number of tokens the transformer must process, with a strong, direct effect on computational cost (self-attention's cost grows quadratically with the number of tokens); and unlike a CNN, which is architecturally translation-invariant by construction (Day 26 §6.2), a ViT must learn positional relationships from data, making the specific choice of position-embedding scheme (fixed at a given resolution, or explicitly designed to interpolate to new resolutions) a preprocessing-adjacent design decision with no real analogue in the classical CNN preprocessing pipelines this day has focused on.

Chapter Summary: Key Takeaways

- **Preprocessing is fixed and deterministic; augmentation is stochastic and training-time only** (§1.2) — conflating the two, or applying augmentation at inference time, is a fundamental design error.
- **Interpolation method matters, and masks must always use nearest-neighbor** (§2.1–2.2) — a rule that becomes directly relevant again in Day 33.
- **Filtering is convolution** (§4.1) — the same sliding-window operation as Day 26's Sobel kernels and Day 29's learned CNN layers, differing only in where the kernel weights come from.
- **Normalization statistics must be fixed and identical between training and inference** (§5.2) — the single most common silent failure mode covered in this day.
- **Augmentation must respect domain-specific constraints** (§6.4) — an augmentation that changes what the label describes is not a valid augmentation, regardless of how standard it is in other domains.
- **GPU-accelerated preprocessing** (§8) becomes worthwhile once CPU preprocessing bottlenecks GPU training throughput — exactly the comparison this day's assignment, PP19, asks you to measure directly rather than assume.

Assignment / Practical Project Brief

PP19: CUDA Accelerated Image Processing

This project asks you to implement the same preprocessing pipeline (resize, normalize, augment) using all three GPU-accelerated approaches from Section 8.3, and to benchmark all three against a CPU baseline — the goal is a genuine, measured comparison on your own hardware, not a repetition of the illustrative numbers in Figure 5.

A Approach A — torchvision.transforms.v2 (GPU-Native)

The highest-level, easiest-integration option: tensor-based transforms executed directly on-device, requiring the least change to an existing PyTorch training loop of the three approaches.

```
import torchvision.transforms.v2 as T
import torch

device = "cuda"
gpu_transform = T.Compose([
    T.RandomResizedCrop(224),
    T.RandomHorizontalFlip(p=0.5),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

# Move the raw batch to GPU BEFORE transforming -- this is what makes it GPU-
# native
batch = raw_batch.to(device, non_blocking=True)
processed = gpu_transform(batch)
```

B Approach B — NVIDIA DALI

The purpose-built, production-realistic data-loading pipeline from Section 8.3, expected to be the steepest of the three to set up but the best-performing at scale, since it overlaps preprocessing with training compute rather than merely running transforms on the GPU.

```
from nvidia.dali import pipeline_def, fn, types
from nvidia.dali.plugin.pytorch import DALIGenericIterator

@pipeline_def(batch_size=256, num_threads=4, device_id=0)
def dali_pipeline(file_root):
    jpegs, labels = fn.readers.file(file_root=file_root, random_shuffle=True)
    images = fn.decoders.image(jpegs, device="mixed") # decode directly on GPU
    images = fn.random_resized_crop(images, size=224)
    images = fn.crop_mirror_normalize(
        images,
        mean=[0.485*255, 0.456*255, 0.406*255],
        std=[0.229*255, 0.224*255, 0.225*255],
        mirror=fn.random.coin_flip(probability=0.5),
    )
    return images, labels
```



```
pipe = dali_pipeline(file_root="./train_data")
loader = DALIGenericIterator(pipe, ["images", "labels"])
```

Notice that DALI decodes JPEGs directly on the GPU (`device="mixed"`), a stage that both torchvision-GPU and CuPy typically leave on the CPU — this is one concrete source of DALI's throughput advantage at scale, since JPEG decoding is itself a non-trivial computational cost that Section 8.1's CPU-bottleneck discussion applies to just as much as resizing or normalization.

C Approach C — CuPy (Low-Level)

The NumPy-compatible, array-level approach from Section 8.3 — most educational for understanding what "GPU-accelerated" actually means mechanically, least convenient for assembling a full pipeline, since higher-level operations must be built up from individual array operations rather than called from a pre-built transform catalogue.

```
import cupy as cp
import cupyx.scipy.ndimage as cundi

# Move a batch of images to GPU memory as a CuPy array
batch_gpu = cp.asarray(batch_numpy) # shape: (N, H, W, C)

# Manual normalization -- every step explicit, nothing hidden
mean = cp.array([0.485, 0.456, 0.406]).reshape(1, 1, 1, 3)
std = cp.array([0.229, 0.224, 0.225]).reshape(1, 1, 1, 3)
normalized = (batch_gpu / 255.0 - mean) / std

# Resize must be implemented via a lower-level op (e.g. ndimage.zoom)
# -- unlike torchvision/DALI, there is no built-in "Resize" transform
resized = cundi.zoom(normalized, (1, 0.5, 0.5, 1), order=1)
```

This manual explicitness is precisely CuPy's pedagogical value and its practical cost at once: every operation is visible and inspectable at the array level, which is exactly why Section 8.3 describes it as the most transparent of the three approaches, but a full augmentation pipeline comparable to Approach A's few-line Compose block requires substantially more code to assemble from these lower-level primitives.

D Deliverables, Rubric, and Pitfalls

Implement the identical preprocessing pipeline (resize, normalize, augment) in all three approaches, benchmark the CPU baseline against each GPU approach across at least three different batch sizes (per Section 8.4's point that the GPU advantage is batch-size-dependent), and produce a comparison table/plot of throughput, latency, and setup complexity alongside a short written recommendation for when each approach is the right choice.

Rubric Component	Weight	What's Assessed
All three approaches implemented	35%	Correct, working pipeline in torchvision-GPU, DALI, and CuPy
Benchmark rigor	25%	Multiple batch sizes tested; fair, consistent methodology across approaches

Rubric Component	Weight	What's Assessed
Comparison table/plot & analysis	25%	Clear throughput/latency comparison; sound written recommendation
Code quality & write-up	15%	Clean, documented code; reproducible results

Common pitfalls: data transfer overhead negating GPU gains for small batch sizes (Section 8.4) — report this explicitly rather than omitting small-batch results that make the GPU approaches look worse; DALI's comparatively steep learning curve, which can consume disproportionate assignment time if left until the last session — start with Approach A to have a working baseline before attempting DALI; and CuPy pitfalls with accidentally mixing CPU (NumPy) and GPU (CuPy) arrays in the same operation, which raises confusing errors unless arrays are explicitly and consistently kept on-device throughout the pipeline.

E Submission Checklist

- All three approaches (torchvision-GPU, DALI, CuPy) implemented and producing correct output
- CPU baseline implemented for comparison
- Benchmarks run across at least three batch sizes for every approach
- Comparison table/plot with your own measured numbers, not placeholders
- Written recommendation: which approach fits which deployment scenario, and why
- Code is documented, and a short README explains how to reproduce your results

End of Day 27 — ready for gap review and expansion into full compendium.